

네트워크 기능 구현

XmlPullParser

Server <-- 요청 -- Bitmap, txt
-- inputStream -->



목차

 OpenAPI 사용

 XML Parsing

 Parser 의 사용

Open API

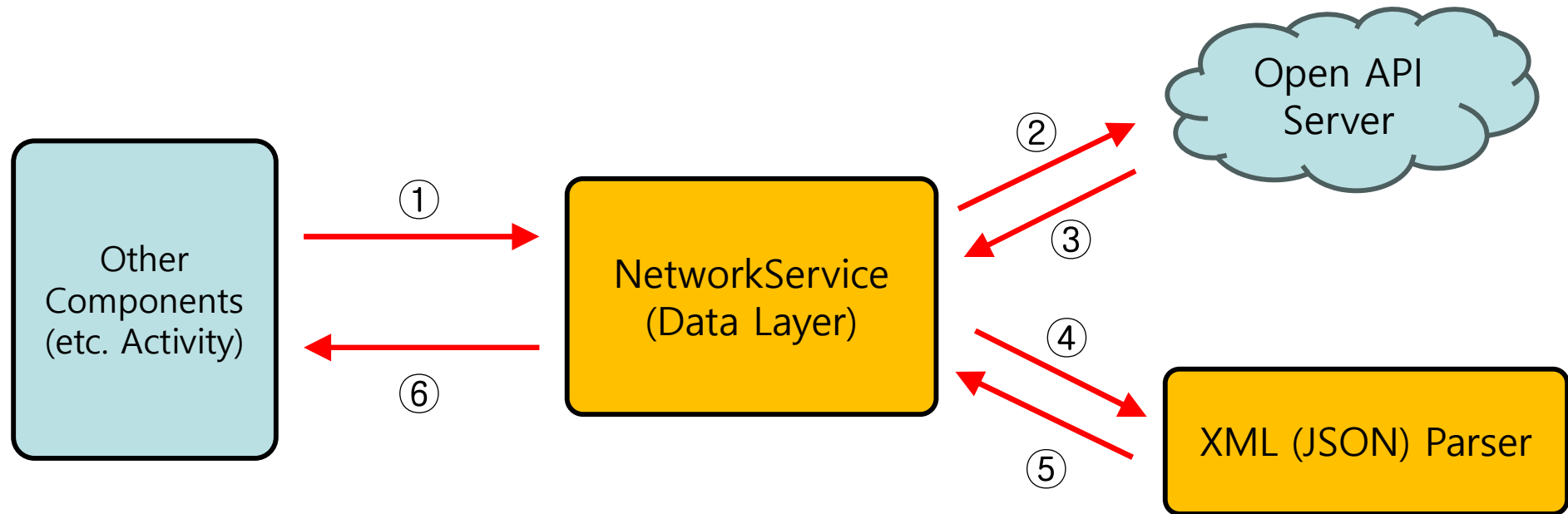
개요

- ◆ Open Application Programming Interface
 - API: 플랫폼이나 웹서비스에서 갖고 있는 기능을 애플리케이션을 구현할 때 사용할 수 있도록 제공하는 인터페이스
- ◆ 일반적으로 REST 형식으로 제공
 - HTTP메소드 요청(GET 등)을 사용하여 리소스의 URI에 접근하고, XML 또는 JSON 등의 형식으로 결과를 받음 요청 param
 - Open API 접근 식별 유형
 - 고유의 key 를 부여하여 URL 매개변수로 전송 영화진흥위원회 OpenAPI
 - Client ID/Client Secret 을 사용하여 Header에 기록보안이 더 좋음(e.g. 네이버)

Open API 서비스 예:

- ◆ 영화진흥위원회 Open API: <https://www.kobis.or.kr/kobisopenapi/homepg/main/main.do>
- ◆ Naver Open API 사용: <https://developers.naver.com/main/>
- ◆ Google Open API 사용 : <https://console.cloud.google.com/>

Open API 처리 절차 및 기본 구조



- ① OpenAPI 요청에 필요한 정보(검색어 등)를 전달하며 처리 요청
- ② OpenAPI 요청형식(URI 및 헤더정보) 정보 구성 후 서비스 요청
- ③ 요청을 처리하여 응답 전달 (XML 또는 JSON 응답)
- ④ 요청 결과 InputStream(또는 String)을 XML(JSON) Parser 에 전달
- ⑤ Parsing 수행 후 각 항목을 DTO 로 변환하여 List<DTO> 형태로 반환
- ⑥ List<DTO> 결과를 요청한 구성요소에 반환

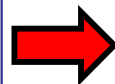
XML Parser 의 구현 예 1

XML 문서를 읽어 하나의 요소를 하나의 DTO 에 저장하는
작업 반복 String 또는 InputStram

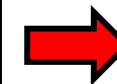
생성한 DTO 는 List 등의 컬렉션에 저장

```

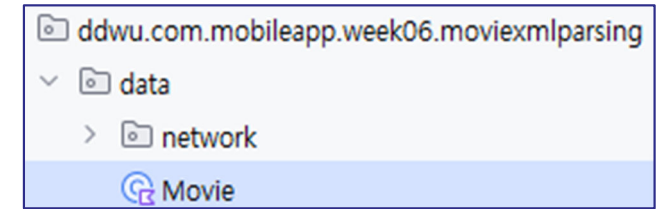
<boxOfficeResult>
  <boxofficeType>일별 박스오피스</boxofficeType>
  <showRange>20120101~20120101</showRange>
  <dailyBoxOfficeList>
    <dailyBoxOffice>
      <rnum>1</rnum>
      <rank>1</rank>
      <rankInten>0</rankInten>
      <rankOldAndNew>OLD</rankOldAndNew>
      <movieCd>20112207</movieCd>
      <movieNm>미션임파서블:고스트프로토콜</movieNm>
      <openDt>2011-12-15</openDt>
      <salesAmt>2776060500</salesAmt>
      <salesShare>36.3</salesShare>
      <salesInten>-415699000</salesInten>
      <salesChange>-13</salesChange>
      <salesAcc>40541108500</salesAcc>
      <audiCnt>353274</audiCnt>
      <audiInten>-60106</audiInten>
      <audiChange>-14.5</audiChange>
      <audiAcc>5328435</audiAcc>
      <scrnCnt>697</scrnCnt>
      <showCnt>3223</showCnt>
    </dailyBoxOffice>
    <dailyBoxOffice>...</dailyBoxOffice>
    <dailyBoxOffice>...</dailyBoxOffice>
    <dailyBoxOffice>...</dailyBoxOffice>
  </dailyBoxOfficeList>
</boxOfficeResult>
  
```



XML Parser



아이템 저장을 위한 DTO 정의



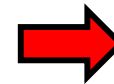
•아이템 저장을 위한 DTO

Data 클래스 생성

```
data class Movie(  
    var rank: Int?,  
    var title: String?,  
    var openDate: String?  
)
```

필요할 경우 _id 등의
필드 추가

DB와 같이 사용할 경우
Entity로 선언



하나의 항목을
표현하는 정보

```
<boxOfficeResult>  
  <boxofficeType>일별 박스오피스</boxofficeType>  
  <showRange>20120101~20120101</showRange>  
  <dailyBoxOfficeList>  
    <dailyBoxOffice>  
      <rnum>1</rnum>  
      <rank>1</rank>  
      <rankInten>0</rankInten>  
      <rankOldAndNew>OLD</rankOldAndNew>  
      <movieCd>20112207</movieCd>  
      <movieNm>미션임파서블:고스트프로토콜</movieNm>  
      <openDt>2011-12-15</openDt>  
      <salesAmt>2776060500</salesAmt>  
      <salesShare>36.3</salesShare>  
      <salesInten>-415699000</salesInten>  
      <salesChange>-13</salesChange>  
      <salesAcc>40541108500</salesAcc>  
      <audiCnt>353274</audiCnt>  
      <audiInten>-60106</audiInten>  
      <audiChange>-14.5</audiChange>  
      <audiAcc>5328435</audiAcc>  
      <scrnCnt>697</scrnCnt>  
      <showCnt>3223</showCnt>  
    </dailyBoxOffice>  
    <dailyBoxOffice>...</dailyBoxOffice>  
    <dailyBoxOffice>...</dailyBoxOffice>  
    <dailyBoxOffice>...</dailyBoxOffice>
```

- XML 문서의 한 항목 정보 → 하나의 DTO 객체에 저장.
- 한 항목이 포함한 태그(속성) → DTO 객체의 하나의 멤버변수에 저장
- 하나의 XML 문서 내의 모든 항목들 → List 등 하나의 컬렉션에 저장

XMLPullParser 주요 요소 (객체명이 parser 일 경우)

XML 파서

-DOM

-SAX

-XMLPullParser

parser.name

- ◆ getName() 의 반환 값, TAG의 이름 반환

parser.eventType

- ◆ getEventType()의 반환 값, 현재 읽은 항목에 따른 유형 반환
START_TAG, END_TAG, TEXT 등

parser.text

- ◆ getText() 의 반환 값, TAG 사이의 Text 값

parser.next()

- ◆ XML 문서의 다음 항목으로 이동

parser.nextTag()

- ◆ XML 문서의 다음 TAG 로 이동. TAG 가 아닐 경우 예외 발생

parser.require(TAG_Type, namespace, TAG)

- ◆ TAG의 유형 및 종류 확인

XmlPullParser의 생성

```
class BoxOfficeParser {
    private val ns: String? = null
```

• namespace 사용 시 적용
→ 예제에서는 사용 안함

```
companion object {
    val FAULT_RESULT = "faultResult" // OpenAPI 결과에 오류가 있을 때에 생성하는 정보를 위해 지정
    val UPPER_TAG = "dailyBoxOfficeList"
    val DAILY_BOXOFFICE_TAG = "dailyBoxOffice"
    val RANK_TAG = "rank"
    val TITLE_TAG = "movieNm"
    val OPEN_DATE_TAG = "openDt"
}
```

• Parsing 대상 태그들을 정의

• 예외 사항 발생 시
Throws

```
@Throws(XmlPullParserException::class, IOException::class)
fun parse(inputStream: InputStream?) : List<Movie> {
```

• use 인라인함수
블록 수행 후 해당 자원 close

use는 →
코틀린의 범위함수
(close 자동 호출함)

```
    inputStream.use { inputStream ->
        val parser : XmlPullParser = Xml.newPullParser()
        /*Parser의 동작 정의, next() 호출 전 반드시 호출 필요*/
        parser.setFeature(XmlPullParser.FEATURE_PROCESS_NAMESPACES, false)
```

```
        /*Parsing 대상이 되는 inputStream 설정*/
        parser.setInput(inputStream, null)
```

```
        /*Parsing 대상 태그의 상위 태그까지 이동*/
```

```
        while (parser.name != UPPER_TAG) {
            parser.next()
        }
```

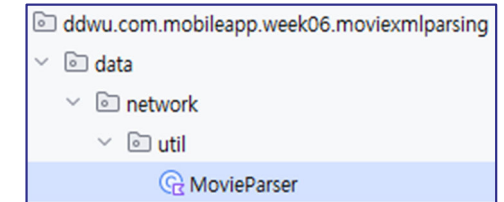
• parser.next()
다음 항목으로 이동

```
        return readBoxOffice(parser)
    }
```

• 세부 Parsing 함수
구현 필요

• parser.name
현재 항목이 태그일
경우 태그 이름 반환

```
    }
    ... // 추가 부분
}
```



• Parsing 대상 ITEM의
상위 태그 지정

```
<boxOfficeResult>
  <boxofficeType>일별 박스오피스</boxofficeType>
  <showRange>20120101~20120101</showRange>
  <dailyBoxOfficeList>
    <dailyBoxOffice>
      ...
    </dailyBoxOffice>
    <dailyBoxOffice>
      <rnum>2</rnum>
      <rank>2</rank>
    </dailyBoxOffice>
  </dailyBoxOfficeList>
</boxOfficeResult>
```


대상 항목 탐색 XML Movie를 읽어서 --> XML DTO를 만들어서 --> List에 담아서 반환한다

```
@Throws(XmlPullParserException::class, IOException::class)
private fun readBoxOffice(parser: XmlPullParser) : List<Movie> {
```

```
    val movies = mutableListOf<Movie>()
```

• DTO 를 저장할
List 생성

• 현재 TAG를 확인
아닐 경우 예외 발생

```
    parser.require(XmlPullParser.START_TAG, ns, "dailyBoxOfficeList")
```

```
    while(parser.next() != XmlPullParser.END_TAG) {
        if (parser.eventType != XmlPullParser.START_TAG) {
            continue
```

• 현재 Parser 가 읽은
유형의 종류 확인

```
        }
        if (parser.name == DAILY_BOXOFFICE_TAG) {
            movies.add(readDailyBoxOffice(parser))
        } else {
            skip(parser)
        }
    }
```

• 항목을 표현하는 TAG
를 찾았을 경우 세부
내용 Parsing

• 대상 태그가 아닐 경우
다음으로 이동

```
    return movies
}
```

• 생성한 List<DTO>
반환

```
<boxOfficeResult>
  <boxofficeType>일별 박스오피스</boxofficeType>
  <showRange>20120101~20120101</showRange>
  <dailyBoxOfficeList>
    <dailyBoxOffice>
      ...
    </dailyBoxOffice>
    <dailyBoxOffice>
      <rnum>2</rnum>
      <rank>2</rank>
    </dailyBoxOffice>
  </dailyBoxOfficeList>
</boxOfficeResult>
```

세부 parsing 함수 2

☞ 항목의 관심 태그 부분 parsing

```

@Throws(XmlPullParserException::class, IOException::class)
private fun readDailyBoxOffice(parser: XmlPullParser) : Movie {
    parser.require(XmlPullParser.START_TAG, ns, "dailyBoxOffice")
    var rank : Int? = null
    var title : String? = null
    var openDate: String? = null

    while (parser.next() != XmlPullParser.END_TAG) {
        if (parser.eventType != XmlPullParser.START_TAG) {
            continue
        }
        when (parser.name) {
            RANK_TAG -> rank = readTextInTag(parser, RANK_TAG).toInt()
            TITLE_TAG -> title = readTextInTag(parser, TITLE_TAG)
            OPEN_DATE_TAG -> openDate = readTextInTag(parser, OPEN_DATE_TAG)
            else -> skip(parser)
        }
    }
    return Movie (rank, title, openDate)
}
    
```

• 항목의 각 값을 저장할 변수 선언

• 관심 TAG를 찾았을 경우 태그 사이의 TEXT 확인

• 필요할 경우 형변환

• 관심 TAG가 아닐 경우 다음으로 이동

• 필요한 정보를 다 읽은 후 DTO 로 저장하여 반환

세부 parsing 함수 3

TEXT 읽기 함수

```
@Throws(IOException::class, XmlPullParserException::class)
private fun readTextInTag (parser: XmlPullParser, tag: String): String {
    parser.require(XmlPullParser.START_TAG, ns, tag)
    var text = ""
    if (parser.next() == XmlPullParser.TEXT) {
        text = parser.text
        parser.nextTag()
    }
    parser.require(XmlPullParser.END_TAG, ns, tag)
    return text
}
```

• 현재 Text 읽기

• 다음 TAG 읽기, TAG 가 아닐 경우 예외 발생

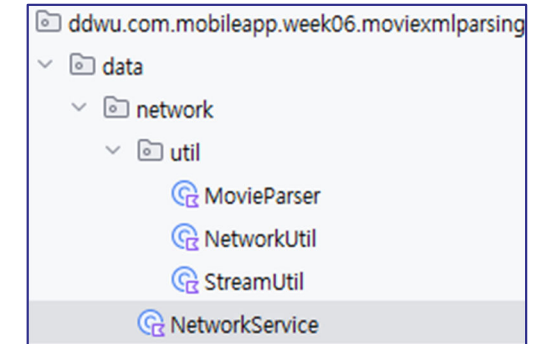
XML문서 이상있는지 체크

다음 TAG 로 넘어가기 함수

```
@Throws(XmlPullParserException::class, IOException::class)
private fun skip(parser: XmlPullParser) {
    if (parser.eventType != XmlPullParser.START_TAG) {
        throw IllegalStateException()
    }
    var depth = 1
    while (depth != 0) {
        when (parser.next()) {
            XmlPullParser.END_TAG -> depth--
            XmlPullParser.START_TAG -> depth++
        }
    }
}
```

Parser 의 사용

NetworkService 에서 호출



```
class NetworkService(private val context: Context) {
    suspend fun getDailyBoxOffice(date: String) : List<Movie> {
        val address = context.resources.getString(R.string.movie_url)

        val params = HashMap<String, String>()
        params["key"] = context.resources.getString(R.string.movie_key)
        params["targetDt"] = date

        val result: InputStream? = try {
            NetworkUtil(context).sendRequest(NetworkUtil.GET, address, params)
        } catch (e: Exception) {
            e.printStackTrace()
            null
        }

        return MovieParser().parse(result)
    }
}
```

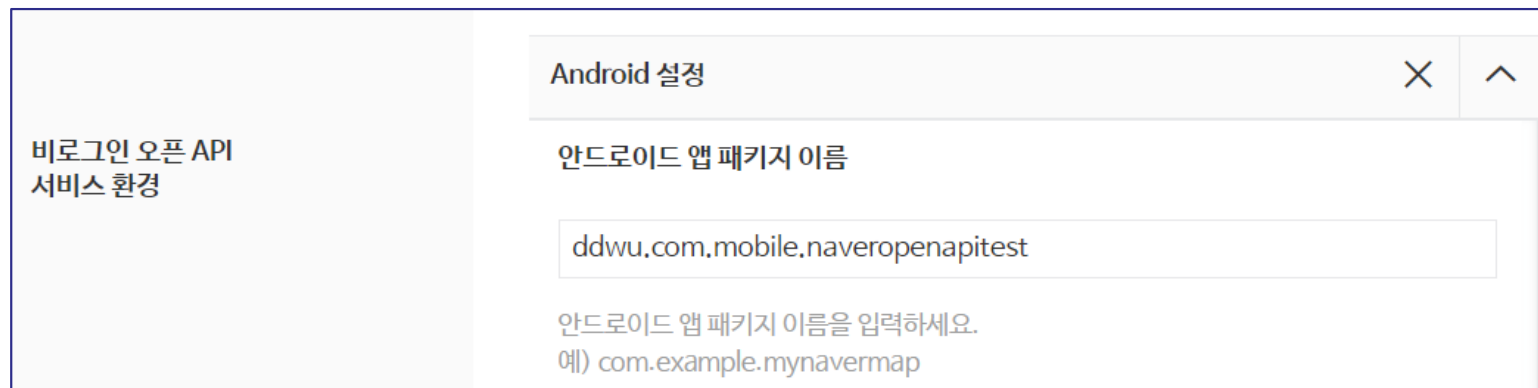
• HashMap<String, String> 을 사용하여 전달할 parameter 를 key, value 의 쌍으로 저장

• Request Method방식, OpenAPI서버주소, 매개변수를 전달하여 결과를 InputStream 으로 반환받음

• OpenAPI 서버로부터 수신한 InputStream 을 Parser에 전달 → List<Movie> 반환

네이버 오픈 API 의 검색 결과를 Parsing 하는 앱을 완성하시오.

- ◆ 키워드 입력 시 관련 책 검색 - NaverParsing_sample 사용
- ◆ 네이버 개발자 사이트에서 애플리케이션 등록 후 진행
 - <https://developers.naver.com/products/service-api/search/search.md>
 - Client ID 및 Client Secret 을 발급 받아 strings.xml 에 기록
 - 사용 API 는 [검색] → [책] 선택
 - [API 설정] 탭에서 비로그인 오픈API 서비스 환경은 [안드로이드] 선택 후 프로젝트의 패키지명 기록



실습 참고

❏ Paring 항목

- ◆책 제목(title), 저자(author), 출판사(publisher), 이미지(image)
- ◆RecyclerView 에는 [책제목 (저자)] 형식으로 출력

❏ NetworkUtil 클래스 수정

```
@Throws(IOException::class, SocketTimeoutException::class)
fun sendRequest(requestMethod: String, address: String, data: Map<String, String>?) : InputStream? {
    ...
    return (url.openConnection() as? HttpURLConnection)?.apply {
        readTimeout = 5000
        connectTimeout = 5000
        doInput = true
        this.requestMethod = requestMethod

        /*Naver 인증 정보를 추가*/
        addRequestProperty("X-Naver-Client-Id", /* strings.xml의 Naver 인증정보 추가*/ )
        addRequestProperty("X-Naver-Client-Secret", /* strings.xml의 Naver 인증정보 추가*/ )
    }
}
```

• Naver에서 발급받은 ID와 Secret 값을 strings.xml 에 저장한 후 sendRequest 사용 시 함께 전송

참고

XML 데이터 파싱

- ◆ <https://developer.android.com/training/basics/network-ops/xml?hl=ko>

영화진흥위원회 Open API

- ◆ <https://www.kobis.or.kr/kobisopenapi/homepg/main/main.do>

네이버 Open API 책 검색

- ◆ <https://developers.naver.com/docs/serviceapi/search/book/book.md#%EC%B1%85>

네이버 Application 관리

- ◆ <https://developers.naver.com/apps/#/list>